

Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi^{1*}

^{1*}Division of Computer Science, Department of Applied Mathematics,
Faculty of Mathematical Sciences, Ferdowsi University of Mashhad,
Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN,
ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Accurate traffic forecasting remains challenging due to complex spatial and temporal dependencies. Most graph-based methods rely on a predefined adjacency matrix built from physical road connectivity, even though proximity need not match the dependency structure in the observed speeds. We investigate Graph Structure Learning (GSL) for traffic forecasting: the adjacency is estimated from the data using contemporaneous and multi-lag graphs learned by DAGMA, a differentiable approach to learning directed acyclic graphs, rather than being taken from a physical road-network matrix. We evaluate the learned graphs with GCN and T-GCN on the public SZ-Taxi and Los-loop benchmarks. Relative to the physical adjacency, learned graphs can reduce forecasting error; the largest and most consistent gains on Los-loop occur when lag-specific graphs are used in a temporally aligned way in T-GCN, with up to 43.0% relative RMSE reduction across horizons 1–4. On SZ-Taxi, the learned multi-lag structure is nearly empty and improvements over the physical graph are much smaller. Matched edge-budget controls show that the observed Los-loop improvement over the physical graph cannot be explained by sparsity alone: random and correlation-placed graphs at the same budget are worse, whereas DAGMA-placed lag graphs with lag-aligned use perform better. A graph-free identity baseline is reported as a reference control. Overall, learning the adjacency from traffic data can improve short-horizon forecasting when the recovered structure is informative and is incorporated into the model consistently with how it was estimated. The learned graphs are interpreted as statistical associations, not causal relationships.

Keywords: Traffic Forecasting, Graph Knowledge Discovery, Graph Structure Learning, Graph Convolutional Networks, T-GCN, Multi-Lag Graph Learning

1 Introduction

Recent advances in graph knowledge discovery have opened new possibilities for understanding complex urban systems, particularly in transportation networks. This paper applies the principles of graph knowledge discovery to traffic prediction, and examines whether learning the graph structure from traffic data improves forecasting relative to graphs that are predefined from physical connectivity. In recent years, traffic prediction has garnered significant attention in smart urban transportation systems. In navigation, ride-hailing, and route planning, accurate traffic prediction plays a crucial role in reducing congestion and increasing the efficiency of transportation networks. Traffic prediction involves analyzing traffic conditions, including flow and speed, extracting traffic patterns, and forecasting traffic trends on urban roads. Due to the complex spatial and temporal dependencies involved, this problem remains challenging and is an active area of research.

The spatial dependencies in this problem arise from the fact that traffic conditions on one road segment are statistically related to conditions on other segments through the topology of the urban road network and through shared demand patterns. Traffic volume also changes dynamically over time, affected by short-term and long-term patterns, which underscores the temporal dependencies in traffic data.

These relationships motivate sophisticated models that can capture joint spatio-temporal structure. A critical limitation of many existing approaches, however, is their reliance on physical proximity or predefined road-network topology to construct the adjacency matrix. Physical connectivity need not coincide with functional co-movement in the data: distant roads can exhibit strong traffic correlations because of commuting patterns, while adjacent roads can have weak statistical dependence. Moreover, dense or otherwise mismatched proximity graphs may degrade graph-based forecasting, for example by inducing excessive neighborhood mixing or an unsuitable inductive bias. In such cases, incorporating a predefined graph may provide little benefit or may even perform worse than the corresponding graph-free backbone. A bibliometric analysis of 784 traffic-forecasting articles (2000–2025) shows that graph-based methods have become a dominant recent focus while many systems still construct the adjacency heuristically from road topology (Appendix B). These observations raise a natural question: if the adjacency matrix is a modeling choice rather than a fixed physical fact, should it be estimated from the traffic data themselves?

Multiple traffic prediction methods, including neural networks, statistical approaches, and Bayesian methods, model temporal evolution but neglect spatial dependencies [1]. To describe spatial features, many studies use graph-based methods [2–5]. In [6], a Temporal Graph Convolutional Network (T-GCN) was proposed for traffic prediction. T-GCN integrates graph-based spatial aggregation into a recurrent gated architecture and achieves strong results on standard traffic benchmarks. In the implementation used in this study, graph convolution is incorporated into the gate computations of a T-GCN cell (the formulation in the public T-GCN code lineage), rather than as a standalone GCN followed by an independent vanilla GRU.

Existing graph-based methods typically build the adjacency matrix from road proximity or distance [6]. While such graphs capture geographic connectivity, they need not represent the statistical dependency structure present in the observed speeds.

Attention-based methods [7–9] attempt to address this by dynamically adjusting relationship weights between nodes. However, these approaches operate on a fundamental distinction: they learn *implicit relationship weights* rather than discovering the *explicit graph structure* itself. Attention can indicate that node j is important to node i at a given time step, but it does not by itself reveal a persistent statistical association independent of the initial topology. These methods therefore still depend on an initial (often suboptimal) graph defined by heuristics or physical proximity, and may miss non-local traffic associations that lack explicit spatial connections.

Our work investigates *graph structure learning* (GSL) for traffic forecasting, in which the adjacency structure is inferred from traffic observations rather than specified in advance. Continuous optimization methods for learning directed acyclic graphs, such as NOTEARS [10] and DAGMA [11], provide practical tools for estimating such structures. Unlike methods that retain a fixed topology and only learn implicit edge or feature reweightings, GSL constructs an explicit weighted adjacency matrix that can be inspected and, after thresholding, represented as a binary graph. This graph should be understood as an estimator-dependent representation of statistical dependencies or associations among sensors, rather than as a direct description of physical connectivity. Importantly, learning an adjacency structure does not by itself guarantee improved forecasting. We therefore evaluate the learned graphs against both physical-graph and graph-free references under a common experimental protocol. The learned structures are interpreted descriptively under the assumptions and constraints of the estimator, not as evidence of causal relationships or physical traffic propagation.

This paper evaluates that idea under a controlled experimental design on two public benchmarks, SZ-Taxi and Los-loop. In addition to the physical road-network graph, we include a graph-free identity control, so that any contribution of learned structure can be measured against both a standard baseline and a model that uses no spatial graph. We consider a single contemporaneous learned graph (and its binary symmetrization, cGSL) as well as *multi-lag* learned graphs in which lag-specific structures are estimated from an explicitly lag-stacked input. Because a learned adjacency can be applied as one static operator or aligned with input timesteps in a recurrent backbone, we further examine how the learned multi-lag structure is incorporated into the forecasting backbone. Finally, matched-sparsity controls address the methodological concern that any gain might be due only to the sparsity of the learned graph rather than to the placement of its edges.

The experimental results show that learning graph structure from traffic data can reduce forecasting error relative to the physical road-network graph. On Los-loop, the largest and most consistent gains occur when lag-specific graphs are used in a lag-aligned manner in T-GCN (up to 43.0% relative RMSE reduction versus Physical across PH1–PH4). On SZ-Taxi, where the recovered multi-lag structure is nearly empty, the reduction versus Physical is much smaller. Matched edge-budget controls further indicate that the Los-loop gain cannot be explained by sparsity alone. Thus, the usefulness of a learned graph depends on the dataset and on how the learned structure is incorporated into the forecasting model.

The contributions of this paper are the following:

1. We investigate *graph structure learning* (GSL) for traffic forecasting with GCN and T-GCN backbones, estimating an alternative adjacency structure from traffic data instead of relying exclusively on a predefined physical adjacency.
2. We evaluate two types of learned structure: a contemporaneous graph and a multi-lag graph construction.
3. We study how multi-lag graphs are incorporated into the forecasting backbone, comparing static union, lag-aligned recurrent use, and a gated mixing variant. This separates the question of how the structure is estimated from how it is subsequently used by the forecasting model.
4. We report five-seed experiments on the SZ-Taxi and Los-loop benchmarks, together with matched-sparsity controls on Los-loop.. The results show that the forecasting value of learned structure is dataset-dependent and that the improvement over the physical graph on Los-loop cannot be attributed to sparsity alone.
5. We compare the learned-graph models with both the physical-graph baseline and a graph-free baseline, thereby assessing whether the learned structure provides an advantage beyond the predefined topology and whether using a graph is beneficial relative to the corresponding graph-free backbone.

In the following sections, we review background on traffic prediction and GCN/T-GCN models (Section 2). We then present the GSL framework used to estimate the adjacency matrix and its integration with GCN and T-GCN (Section 3). Experimental settings are described in Section 4, results in Section 5, and discussion in Section 6. Limitations are stated in Section 7, and conclusions in Section 8.

2 Background and Related Work

The goal of traffic forecasting is to predict future traffic conditions from historical data. Traffic conditions can include metrics such as speed, flow, or density; in our experiments we focus on traffic speed without loss of generality.

We model the road network as a graph $G = (V, E)$, where $V = \{v_1, \dots, v_N\}$ represents N road segments (sensors) as nodes, E represents edges between connected roads, and $\mathbf{A} \in \{0, 1\}^{N \times N}$ is the adjacency matrix encoding those connections (or, in learned variants, estimated statistical adjacency). The traffic data are represented by a feature matrix $X \in \mathbb{R}^{N \times P}$, where P is the length of the historical series and $X_t \in \mathbb{R}^N$ is the vector of speeds at time t . The forecasting task is to learn a mapping

$$[X_{t+1}, \dots, X_{t+T}] = f(G; [X_{t-n}, \dots, X_t]), \quad (2.1)$$

where n is the number of historical steps used as input and T is the prediction horizon.

2.1 Overview of Traffic Prediction Approaches

Traffic prediction methods can be broadly categorized as follows.

Classical prediction methods include ARIMA models [12, 13], support vector regression [14], seasonal ARIMA variants [15], and Kalman filtering approaches [16]. These

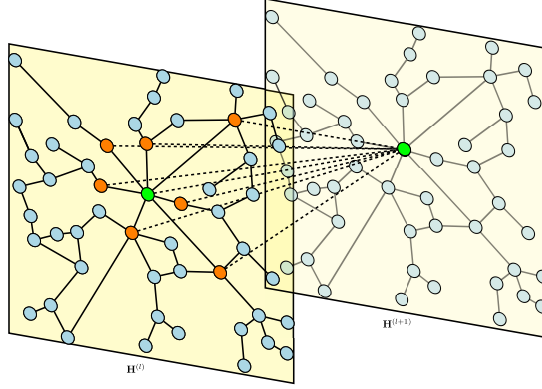


Fig. 1: A graph convolutional architecture with layers l and $(l+1)$ computing $\mathbf{H}^{(l+1)}$ as in (2.3). Neighbors are defined by the adjacency matrix $\mathbf{A}$. In the standard setting $\mathbf{A}$ comes from the physical road network; in this work adjacency is also estimated from traffic data.

methods are effective for basic temporal patterns on individual series but often struggle with network-scale spatial dependence and strong nonlinearity.

Classical deep learning methods include feedforward networks [17], spatio-temporal residual networks [18], interactive temporal recurrent convolutional networks [19], and recurrent variants such as GRU and LSTM [20, 21]. They capture rich temporal structure and some spatial coupling, but their Euclidean assumptions limit the modeling of irregular network topology.

Graph-based and temporal graph methods explicitly model network topology. Representative models include GCNs [22, 23], T-GCN [6], attention-based temporal GCNs (A3T-GCN) [7], dynamic spatial-temporal attention networks [8], and a range of recent spatio-temporal graph convolutional, recurrent, and transformer architectures [9, 24–28]. These methods model topology and time jointly, but they typically rely on a predefined graph that may not match the dependency structure in the data.

Comprehensive reviews are available elsewhere [26, 29, 30]. Our focus is the role of the adjacency matrix itself (motivated in the Introduction and Appendix B). Since this paper builds on GCN and T-GCN, we summarize those backbones only as far as needed for the experiments; fuller treatments appear in [6, 22].

2.2 Graph Convolutional Network

Graph convolutional networks (GCNs) extend convolutional operations to non-Euclidean domains represented by graphs. In traffic forecasting, the graph nodes correspond to traffic sensors, while the graph edges represent the relationships used to propagate information between sensors. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A})$ denote a graph with $N = |\mathcal{V}|$ nodes and adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$. By adding self-loops, the adjacency matrix is defined as

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad (2.2)$$

where $\mathbf{I}$ is the identity matrix. Let $\tilde{\mathbf{D}}$ be the corresponding degree matrix. A standard GCN layer is then expressed as

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (2.3)$$

where $\mathbf{H}^{(l)}$ is the node-feature matrix at layer l , $\mathbf{W}^{(l)}$ is a trainable feature-transformation matrix, and $\sigma(\cdot)$ is a nonlinear activation function. The normalized adjacency operator aggregates information from neighboring nodes and therefore provides a mechanism for spatial information propagation.

In traffic forecasting, the input associated with each sensor is often a historical sequence of observed speeds rather than a single scalar. Let B denote the batch size and T the length of the historical observation window. The input tensor can consequently be represented as

$$\mathbf{X} \in \mathbb{R}^{B \times T \times N}, \quad (2.4)$$

where $\mathbf{X}_{b,t,n}$ is the observed speed at node n , time t , and batch sample b . For a particular sample b , the historical window can be viewed as a node-feature matrix $\mathbf{X}_b^T \in \mathbb{R}^{N \times T}$, in which the T historical observations constitute the feature vector of each node.

Accordingly, a graph-convolution operation on the historical input can be written in the form

$$\mathbf{Z}_b = \sigma \left(\mathbf{G} \mathbf{X}_b^T \mathbf{W} \right), \quad \mathbf{G} \in \mathbb{R}^{N \times N}, \quad \mathbf{W} \in \mathbb{R}^{T \times d}, \quad (2.5)$$

where $\mathbf{G}$ denotes the graph-propagation operator constructed from the adjacency matrix, d is the output feature dimension, and $\mathbf{W}$ is shared across all graph nodes. The resulting matrix $\mathbf{Z}_b \in \mathbb{R}^{N \times d}$ contains spatially aggregated representations of the historical observations.

This formulation emphasizes that the graph operation models relationships between nodes, whereas the historical window supplies the input features associated with each node. The graph convolution itself does not explicitly model the temporal evolution of these features; temporal dependencies must either be represented through the input window or learned by an additional temporal component.

2.3 Temporal Graph Convolutional Network

The Temporal Graph Convolutional Network (T-GCN) proposed by Zhao et al. [6] combines graph-based spatial information propagation with a recurrent gated architecture for traffic forecasting. Its purpose is to jointly capture dependencies among spatially related traffic sensors and the temporal evolution of traffic conditions.

Unlike a model that first applies a standalone GCN and subsequently passes its output to an independent vanilla GRU, the T-GCN architecture incorporates graph convolution directly into the recurrent cell. More specifically, graph convolution is applied to the concatenation of the current input and the previous hidden state. The resulting graph-aggregated representation is then used in the gate and candidate computations of the recurrent unit.

Let $\mathbf{x}_t \in \mathbb{R}^N$ denote the traffic observations at time t , and let $\mathbf{h}_{t-1} \in \mathbb{R}^{N \times d_h}$ denote the previous hidden state. The input and hidden state are concatenated along the

feature dimension and processed by a graph-convolution operator. In abstract form, this operation can be represented as

$$\mathbf{S}_t = \text{GCN}([\mathbf{x}_t, \mathbf{h}_{t-1}]), \quad (2.6)$$

where $[\cdot, \cdot]$ denotes feature-wise concatenation. The resulting spatially aggregated representation $\mathbf{S}_t$ is used to compute the update gate, reset gate, and candidate hidden state. In a generic gated formulation, these operations can be written as

$$\mathbf{z}_t = \sigma(\mathbf{S}_t^{(z)}), \quad (2.7)$$

$$\mathbf{r}_t = \sigma(\mathbf{S}_t^{(r)}), \quad (2.8)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{S}_t^{(h)}), \quad (2.9)$$

where $\mathbf{z}_t$ and $\mathbf{r}_t$ are the update and reset gates, respectively, and $\tilde{\mathbf{h}}_t$ is the candidate hidden state. The hidden state is subsequently updated according to the GRU mechanism,

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \quad (2.10)$$

where $\odot$ denotes element-wise multiplication. The graph convolution in these computations allows the recurrent unit to account for spatial interactions among sensors, while the recurrent state provides a mechanism for retaining information over successive time steps.

It is important to distinguish the two types of dependencies modeled by T-GCN. The graph operator describes how information is propagated between nodes and therefore represents the spatial component of the model. In contrast, the recurrent hidden state and the gated update mechanism represent the temporal component by carrying information from previous time steps to the current step. Thus, the graph used by T-GCN should not be interpreted as a direct representation of temporal edges or explicit time-lag relationships; rather, it defines the spatial information-propagation structure used within the recurrent computation.

In the public T-GCN code lineage and in the implementation used in this study, the graph-convolution operation is integrated into the T-GCN cell and is applied to the current input together with the previous hidden state during the gate and candidate computations. Therefore, the implementation follows the recurrent graph-convolutional design of T-GCN rather than a pipeline consisting of an independent GCN followed by a conventional GRU. The T-GCN model serves as the principal spatiotemporal backbone in our experiments, whereas the non-recurrent GCN model is used as a spatial counterpart.

2.4 Graph Structure Learning

Graph structure learning estimates $\mathbf{A}$ (or a weighted coefficient matrix W) from observed data rather than from a predefined map [36]. In traffic, the motivation is that proximity is an imperfect proxy for functional co-movement, and that

sparse data-driven graphs may avoid the oversmoothing induced by dense physical adjacencies.

Score-based continuous DAG learners replace combinatorial search over graphs with smooth optimization on real matrices. NOTEARS [10] introduces a differentiable acyclicity constraint; DAGMA [11] strengthens the approach with a log-determinant characterization suitable for efficient optimization. We use DAGMA with a linear structural-equation score and an ℓ_1 sparsity penalty. After fitting, a binary adjacency is obtained by thresholding absolute coefficient magnitudes. Acyclicity here is an optimization device; we do not interpret the learned graph as a causal traffic model. The precise contemporaneous and multi-lag constructions, including thresholds and edge budgets, are specified in Section 3.

Compared with attention mechanisms that reweight neighbors on a fixed topology, GSL changes the edge set itself. Compared with purely temporal models, it makes the spatial operator an object of estimation. In this paper we study that estimated operator both as a single contemporaneous graph and as lag-specific blocks, and we evaluate it against physical and graph-free references under the same forecasting protocol.

This work remains a study of GSL for traffic prediction: we ask whether a learned adjacency is preferable to a physical one, and under what conditions. A graph-free control, an explicit multi-lag construction, and an analysis of how that structure is used in the backbone specify those conditions without changing the paper’s identity.

3 The Proposed Method: Estimating the Adjacency Matrix with Graph Structure Learning

In both GCN [22] and T-GCN [6], the adjacency matrix $\mathbf{A}$ that describes spatial relationships among sensors is typically predefined from urban planning or road-network information. Here we estimate $\mathbf{A}$ from traffic data, which is known as *graph structure learning* (GSL) [36]. The goal is to obtain a graph representation that reflects dependency structure present in the observed series rather than structure imposed solely by physical proximity.

3.1 Motivation

In a typical urban traffic network, spatial proximity of roads need not coincide with statistical dependence among their speeds. Distant segments can be strongly correlated through shared demand, while adjacent segments can be weakly related. Propagation delay is also relevant: association between two sensors may appear only at particular temporal lags. Estimating the adjacency directly from the data allows the forecasting model to aggregate from neighbors defined by observed co-movement rather than only by the map.

Figure 2 illustrates a stylized network related to Figure 1. Three points motivate data-driven adjacency: (i) nearby nodes may share more immediate co-movement than distant ones, but this need not hold for every pair; (ii) associations between distant nodes may occur at longer lags; (iii) some physically adjacent links may carry little predictive information for the target sensor. By learning $\mathbf{A}$ from traffic observations,

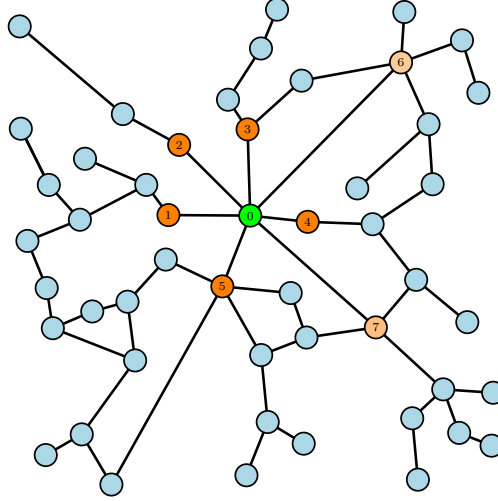


Fig. 2: Conceptual two-dimensional traffic network. Association between a target node (green) and other nodes depends not only on spatial proximity (lighter color indicates larger distance) but also on topology and travel time. The learned adjacency aims to reflect such data-driven associations rather than geographic distance alone.

the procedure estimates edges from the observed data under its modeling assumptions rather than only from geographic distance.

Throughout this section, $\mathbf{A} \in \{0, 1\}^{N \times N}$ denotes the binary adjacency supplied to the forecasting backbone (Eq. (2.3)), and $W \in \mathbb{R}^{d \times d}$ denotes the continuous coefficient matrix estimated by the graph-learning procedure. The two symbols are not interchangeable: W is the estimator’s internal representation; $\mathbf{A}$ is the thresholded graph used by GCN or T-GCN.

3.2 Integration of Learned Structure with Temporal Modeling

The overall pipeline separates three stages:

- Graph estimation (GSL). A static adjacency is estimated from training traffic data (contemporaneous or multi-lag constructions below).
- Normalization. Every adjacency used in this paper — physical, identity (graph-free control), and learned — is passed through the same GCN-style operator

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}}^\top \tilde{\mathbf{D}}^{-1/2}, \quad \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}. \quad (3.1)$$

For the identity adjacency this reduces to $\hat{\mathbf{A}} = \mathbf{I}$, so the graph-free control differs only in the absence of off-diagonal structure.

- Forecasting backbone. GCN or T-GCN uses the normalized operator. The non-recurrent GCN applies a single graph convolution to the flattened input window

(time steps as node features) and a shared linear output head. The T-GCN implementation follows the graph-convolutional gated-cell formulation used in the public T-GCN code lineage: graph convolution is incorporated into the gate computations of the recurrent cell, not as a standalone GCN followed by an independent vanilla GRU. The learned graph is *static* after estimation: it is fitted once on the training split. Temporal variation in the forecast is modeled by the recurrent backbone (and, in one variant, by a gate that selects among fixed lag-specific graphs); the edge set itself does not change over time.

Two reference adjacencies accompany the learned graphs in all experiments: the *physical* road-network graph (standard T-GCN/GCN baseline) and a *graph-free* identity control (T-GCN-NoSpatial / GCN-NoSpatial). The latter is required to measure whether learned structure adds value beyond removing a possibly harmful dense proximity graph.

3.3 Continuous Optimization for Graph Structure Learning

Score-based GSL seeks a weighted adjacency that fits the data under a structural equation model (SEM). Following the continuous formulation of Zheng et al. [10], let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be a data matrix with n observations of d variables. In traffic, rows of $\mathbf{X}$ are time indices and columns are sensors. Instead of searching the discrete DAG space, one optimizes over $W \in \mathbb{R}^{d \times d}$ with a smooth acyclicity constraint [10]:

$$\begin{aligned} \min_W \quad & F(W) \\ \text{subject to} \quad & h(W) = 0, \end{aligned} \tag{3.2}$$

where F measures data fit and h vanishes exactly on acyclic graphs. NOTEARS uses

$$h(W) = \text{tr}(e^{W \circ W}) - d, \tag{3.3}$$

with gradient $\nabla h(W) = (e^{W \circ W})^\top \circ 2W$, and solves the program with an augmented Lagrangian

$$L^\rho(W, \alpha) = F(W) + \frac{\rho}{2} |h(W)|^2 + \alpha h(W). \tag{3.4}$$

After optimization, a binary support is obtained by absolute-magnitude thresholding, $\widehat{W} = W \circ \mathbf{1}(|W| > \omega)$. Algorithmic details appear in [10].

In the experiments we use **DAGMA** [11], which replaces the NOTEARS trace-exponential constraint with a log-determinant acyclicity characterization suitable for efficient constrained optimization, together with a linear SEM score (ℓ_2 reconstruction loss) and an ℓ_1 sparsity penalty $\lambda_1 \|\mathbf{W}\|_1$. Acyclicity is an *optimization regularizer* of the estimator. We interpret the learned graph as a statistical dependency structure under the linear SEM assumptions, not as a causal traffic model. DAGMA is applied in two related ways: a *contemporaneous* construction (single graph among sensors; Section 3.4) and a *multi-lag* construction (lag-specific blocks; Section 3.5).

3.4 Contemporaneous Graph Learning (GSL and cGSL)

The contemporaneous construction is a single learned graph among the N sensors from simultaneous observations — the direct analogue of the original GSL-for-traffic idea. DAGMA is fitted to the training-split snapshot matrix

$$\mathbf{X} = \text{train_norm}[0 :: \text{PH}],$$

i.e. every PH-th training row (one simultaneous vector of N sensor speeds per row; no lag blocks). For Los-loop the row counts are 1612, 806, 538, and 403 for $\text{PH} = 1, \dots, 4$; for SZ-Taxi, 2380, 1190, 794, and 595. The sparsity coefficient is $\lambda_1 = 0.02$ (Los-loop) and $\lambda_1 = 0.01$ (SZ-Taxi). DAGMA’s internal magnitude threshold is 0.3. The binary adjacency is

$$\mathbf{A}_{\text{GSL}} = \mathbf{1}(|W| > 0)$$

after that internal threshold (absolute-magnitude rule; diagonal removed), using training data only. Edge counts are 28 (Los-loop) and 8 (SZ-Taxi) at every horizon.

The cyclic variant of GSL, denoted as *cGSL* (symmetrized GSL) is not a second fit. It is the binary symmetrization of the same stored GSL artifact:

$$\mathbf{A}_{\text{cGSL}} = \mathbf{1}(\mathbf{A}_{\text{GSL}} + \mathbf{A}_{\text{GSL}}^\top > 0), \quad (3.5)$$

with the diagonal removed (56 directed entries on Los-loop, 16 on SZ-Taxi). The same artifacts serve both GCN and T-GCN counterparts, so any difference between backbones is not due to a different learned edge set.

The contemporaneous graph is estimated from simultaneous snapshots. An edge is a statistical association among present observations; it does not encode “sensor j at time t predicts sensor i at time $t + 1$.” Temporal organization of learned structure is introduced explicitly in the multi-lag construction next, rather than inferred from this DAG.

3.5 Multi-Lag Graph Learning

A single contemporaneous graph cannot distinguish associations at different temporal scales. We therefore estimate lag-specific structure from an explicitly lag-stacked input, still within the same DAGMA/GSL framework.

With lag depth $L = 3$,

$$\mathbf{Z} = [x(t-3), x(t-2), x(t-1), x(t)],$$

stacked over training windows (1609 windows on Los-loop; 2377 on SZ-Taxi). The variable dimension is $(L+1)N$: 828 (Los-loop) or 624 (SZ-Taxi). DAGMA is fitted *once* on the training split; the artifact is shared across $\text{PH} = 1, \dots, 4$ (unlike the contemporaneous input, which is PH-subsampled).

Here $\lambda_1 = 0.01$ on both datasets, the internal threshold is 0.0 (raw weights retained), and the forecasting consumer thresholds at $|W| > 0.1$ (absolute magnitude; diagonal removed). In the DAGMA convention, W_{ij} quantifies estimated dependence

of variable j on variable i ; the block mapping lag- k variables to the current-time block yields lag adjacencies $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3$. The contemporaneous block inside W is not consumed by the forecasting methods.

After consumer thresholding, per-lag off-diagonal counts are 12/3/15 (sum 30 slots; union 28 distinct edges) on Los-loop and 0/0/2 (union 2) on SZ-Taxi. The lag-structured reading is consistent with the construction; we do not independently validate it as a physical propagation model, and acyclicity remains a fitting constraint only. The graphs are static after fitting.

3.6 Using Learned Graphs in GCN and T-GCN

Single-graph use (GSL / cGSL / physical / identity).

One normalized adjacency is applied for the whole input window in GCN, and at every recurrent step in T-GCN. This is the standard integration of a learned static structure with temporal modeling: GSL provides persistent spatial dependency estimates; the backbone models temporal evolution on that fixed operator.

Multi-lag use in T-GCN.

When three lag graphs are available, T-GCN can apply them in different ways. We evaluate:

- Fixed lag assignment (T-GCN-MultiGSL). At input index t ($t = 0$ most recent), the graph index is
$$\text{graph_idx} = (T - 1 - t) \bmod 3, \quad (3.6)$$
so recent steps use lag-1 and older steps cycle through lag-2 and lag-3. No extra parameters.
- Global mixture (T-GCN-MultiGSL-Weighted, supplementary). Three learned scalars (softmax) mix the normalized lag operators for every step (+3 parameters).
- Per-node gate (T-GCN-MultiGSL-Mix). At each step t and node, a small MLP on $[x_t; h_{t-1}]$ produces a softmax over the $K = 3$ graphs; the mixed operator is applied before the GRU update (+4,419 parameters). Mix changes how the *static* lag graphs are used per node and timestep; it does not relearn the edge set.

Multi-lag use in GCN (GCN-MultiGSL).

The non-recurrent GCN applies a single operator to the window, so lag graphs are merged into a static union

$$\mathbf{A}_{\cup} = \bigvee_{k=1}^3 \mathbf{A}_k$$

(element-wise OR), then normalized as in (3.1). Comparing GCN-MultiGSL with T-GCN-MultiGSL asks whether the same multi-lag artifacts are useful when consumed as one static graph versus per timestep. Because the backbones also differ, this comparison is evidence about consumption under different architectures, not a fully isolated causal test of consumption alone.

Table 1: Graph sources used in the experiments. All are normalized by Eq. (3.1). “Static” means the adjacency is fixed after estimation; temporal modeling is in the backbone (and in Mix, in the *use* of fixed lag graphs).

Name	Origin	Construction	Use
Physical	road network	dataset adjacency	static
NoSpatial	none	identity $\mathbf{I}$	static
GSL	DAGMA	contemporaneous	static
cGSL	DAGMA	(3.5) of GSL	static
MultiGSL family	DAGMA	lag blocks $\mathbf{A}_{1:3}$	per-step / gate / union

Capacity.

Within a backbone family, graph sources add no trainable parameters. Verified counts (excluding the shared output layer) are 768 for GCN and 12,672 for single-graph T-GCN; Mix adds 4,419 relative to T-GCN-MultiGSL. Capacity-matched checks are reported with the experiments (Section 4).

3.7 Summary of Graph Sources

Table 1 summarizes the adjacencies compared in this paper. The experimental study asks whether replacing the physical graph with a learned one improves forecasting, whether a graph-free control changes that assessment, and — for multi-lag structure — how the learned graphs should be used inside the backbone.

4 Experimental Setup

This section describes the datasets, protocol, model configurations, and evaluation used in our experiments. A preliminary version of the GSL-for-traffic idea was presented in [37].

4.1 Datasets

We use two widely used traffic benchmarks.

SZ-Taxi comprises taxi trajectory speeds in the Luohu District of Shenzhen, China (January 2015), from 156 road sensors aggregated at 15-minute intervals (2976 timesteps).

Los-loop comprises loop-detector speeds on Los Angeles County highways (March 1–7, 2012), from 207 sensors aggregated every 5 minutes (2016 timesteps, seven days).

Each dataset provides a feature matrix of speeds over time and a road-network adjacency used as the physical baseline (Section 3). The two benchmarks contrast urban taxi traffic with highway dynamics.

4.2 Data Split and Normalization

Each series is split chronologically into 80% training and 20% test segments, preserving temporal order: 1612/404 timesteps for Los-loop and 2380/596 for SZ-Taxi. Input windows and targets are generated separately within each segment, so no window crosses the train/test boundary.

Features are normalized by dividing by the *training-split* maximum only (70.0 on Los-loop; 86.4292 on SZ-Taxi). The test split is not used for normalization or for graph learning.

4.3 Forecasting Protocol

The historical window is $T = 12$ steps. Prediction horizons are $PH \in \{1, 2, 3, 4\}$ steps of the native sampling interval: 5–20 minutes on Los-loop and 15–60 minutes on SZ-Taxi. Targets are the multi-step speed vectors produced by a shared linear output layer (Section 3). Horizon indices are comparable within, not across, datasets because the sampling intervals differ.

Following [6], we focus the comparison on GCN and T-GCN as foundational backbones rather than an exhaustive set of recent architectures: the goal is to assess the effect of graph structure learning inside established frameworks that many later models build upon [5, 38].

4.4 Model Configurations

Within each backbone family we compare physical, graph-free, and learned adjacencies, and (for multi-lag graphs) different ways of using the learned structure (Section 3).

T-GCN family.

T-GCN (physical graph); T-GCN-NoSpatial (identity, graph-free control); T-GCN-GSL and T-GCN-cGSL (contemporaneous learned graphs); T-GCN-MultiGSL, T-GCN-MultiGSL-Weighted, and T-GCN-MultiGSL-Mix (multi-lag graphs under fixed, global-mixture, and per-node gating).

GCN family.

GCN (physical); GCN-NoSpatial; GCN-GSL; GCN-cGSL; and GCN-MultiGSL (static union of the multi-lag graphs).

Architectures within a family are identical apart from the adjacency and, for the multi-lag T-GCN variants, the lag-graph use (Section 3.6). Contemporaneous and multi-lag DAGMA hyperparameters and thresholds are as specified in Sections 3.4 and 3.5. For the contemporaneous construction, the sparsity coefficient is $\lambda_1 = 0.01$ on SZ-Taxi and $\lambda_1 = 0.02$ on Los-loop, consistent with the original GSL protocol.

4.5 Training Protocol

All forecasting models are trained with Adam (learning rate 10^{-3} , weight decay 10^{-4}), batch size 128, for 50 epochs, hidden dimension 64, and a shared linear output map from the hidden state to the PH-dimensional target.

The T-GCN family uses an ℓ_2 loss with an ℓ_2 penalty on weights ($\lambda_{\text{reg}} = 1.5 \times 10^{-3}$), matching the reference implementation; the GCN family uses mean squared error.

Each configuration is trained under five seeds $\{42, 43, 44, 45, 46\}$. Learned graphs are fitted once and are deterministic (DAGMA-linear, seed 42); the same graph artifacts are shared across forecasting seeds, so seed variability reflects model training only. Experiments were run in PyTorch on a workstation with an NVIDIA RTX 3090 GPU.

4.6 Evaluation Metrics

We report root mean squared error (RMSE, primary) and mean absolute error (MAE), computed on the de-normalized speed scale over the full test set:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}, \quad (4.1)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|. \quad (4.2)$$

Here n is the total number of evaluated scalar predictions, including all test windows, sensors, and output steps for the selected prediction horizon. For each PH-specific model, all PH target steps are pooled when computing the reported RMSE and MAE (they are not computed separately for the final horizon alone). Additional diagnostics used in the code base (relative accuracy and R^2) are not required for the comparisons below.

4.7 Statistical Reporting

Main-text numbers are five-seed means with sample standard deviations (ddof = 1) over forecasting seeds $\{42, 43, 44, 45, 46\}$. DAGMA graphs are deterministic and shared across these seeds, so seed variability reflects training of the forecasting models, not re-estimation of the graph. We report paired per-seed win counts as descriptive evidence of seed consistency. Paired t -tests are also computed for reference where comparisons are discussed; they are not treated as definitive significance tests. With $n = 5$, the exact Wilcoxon signed-rank test cannot fall below $p = 0.0625$, so we do not claim formal statistical significance. Conclusions rely on effect sizes, five-seed means and spreads, paired consistency, and the stated experimental scope.

4.8 Matched-Sparsity and Capacity Controls

To address the concern that any gain from a learned graph might be due only to sparsity (or to extra parameters of a gated variant), we include two scope-labeled controls on Los-loop at PH = 1.

Matched sparsity. At a 30-edge budget we compare RandTop30 (random directed edges, redrawn per seed), CorrTop30 (top-30 training Pearson correlations), and the DAGMA multi-lag graphs under fixed and gated use, with five seeds under the same training protocol. No sparsified physical graph and no full λ /threshold sweep were run.

Capacity. A T-GCN-NoSpatial model with hidden dimension 74 (16,872 parameters) more closely matches the gated multi-lag model (17,091) than standard T-GCN-NoSpatial at hidden 64 (12,672). This check is single-seed and is labeled as such.

4.9 Ablation Studies

In addition to the main comparison of graph sources, we examine:

- Physical vs. learned vs. graph-free. The physical adjacency is compared with contemporaneous GSL/cGSL and with the identity control, so that learned structure is not credited solely for removing a dense proximity graph.
- GSL vs. cGSL. Directed versus symmetrized use of the *same* contemporaneous artifact on each backbone.
- Multi-lag structure and its use. Fixed lag assignment, global weights, per-node gating, and static union consumption of the same multi-lag artifacts (Section 3.6).
- Temporal resolution. A 15-minute resampling of Los-loop (Section 4.10).

4.10 Temporal-Resolution Variant

Los-loop is additionally averaged over consecutive triplets of 5-minute observations, yielding a 15-minute series of length $T = 672$ with an 80/20 split. Horizons PH = 1, ..., 4 then correspond to 15–60 minutes ahead. This is a resolution variant of Los-loop, not a third dataset, and its PH indices are not equivalent to those of the 5-minute series. A separate multi-lag DAGMA fit is used ($\lambda_1 = 0.01$; consumer threshold $|W| > 0.1$); T-GCN-NoSpatial, T-GCN-MultiGSL, and T-GCN-MultiGSL-Mix are trained with five seeds under the standard protocol.

5 Results

We evaluate whether learning the adjacency from traffic data improves forecasting relative to a physical road-network graph, and under what conditions. Throughout, we call the identity-adjacency control the *graph-free baseline* (T-GCN-NoSpatial / GCN-NoSpatial); it is a reference, not a proposed method. Unless noted otherwise, each value is the mean test RMSE over five training seeds with the sample standard deviation in parentheses; learned graphs are fitted once and shared across seeds. Improvements are relative RMSE reductions with respect to the named reference. Statistical qualifications follow Section 4.7.

Table 2: T-GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. PH1–PH4 correspond to 5–20 min (Los-loop) and 15–60 min (SZ-Taxi). The italic first row in each block is the graph-free control (not a proposed method). Bold: lowest mean among the compared methods only (not a significance claim).

Method	PH1	PH2	PH3	PH4
<i>Los-loop (5 min)</i>				
<i>Graph-free baseline (NoSpatial)</i>	<i>5.25 (0.19)</i>	<i>5.76 (0.08)</i>	<i>6.11 (0.07)</i>	<i>6.58 (0.12)</i>
T-GCN (Physical)	7.88 (0.28)	8.13 (0.09)	8.37 (0.17)	8.66 (0.16)
T-GCN-GSL	5.86 (0.21)	6.30 (0.06)	6.66 (0.12)	7.04 (0.09)
T-GCN-cGSL	5.82 (0.23)	6.35 (0.12)	6.67 (0.11)	7.09 (0.12)
T-GCN-MultiGSL	4.84 (0.11)	5.45 (0.14)	5.94 (0.03)	6.26 (0.03)
T-GCN-MultiGSL-Weighted	4.83 (0.11)	5.44 (0.13)	5.93 (0.03)	6.25 (0.03)
T-GCN-MultiGSL-Mix	4.49 (0.14)	5.08 (0.15)	5.55 (0.16)	5.86 (0.07)
<i>SZ-Taxi (15 min)</i>				
<i>Graph-free baseline (NoSpatial)</i>	<i>4.12 (0.00)</i>	<i>4.16 (0.01)</i>	<i>4.19 (0.01)</i>	<i>4.23 (0.01)</i>
T-GCN (Physical)	5.45 (0.11)	5.55 (0.08)	5.60 (0.05)	5.63 (0.06)
T-GCN-GSL	4.28 (0.05)	4.31 (0.03)	4.33 (0.01)	4.37 (0.02)
T-GCN-cGSL	4.30 (0.03)	4.34 (0.02)	4.38 (0.03)	4.41 (0.01)
T-GCN-MultiGSL	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Weighted	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Mix	4.12 (0.02)	4.15 (0.01)	4.18 (0.00)	4.22 (0.01)

5.1 Comparison with the Physical Road-Network Graph

Table 2 reports the T-GCN family against the physical road-network adjacency. Every learned configuration reduces error relative to Physical in most cells. On Los-loop at PH1, T-GCN (Physical) obtains 7.88 ± 0.28 RMSE, whereas T-GCN-GSL and T-GCN-cGSL obtain 5.86 ± 0.21 and 5.82 ± 0.23 , and the multi-lag variants obtain 4.84 ± 0.11 (MultiGSL) and 4.49 ± 0.14 (Mix). Relative to Physical, Mix reduces RMSE by 43.0%, 37.6%, 33.7%, and 32.3% at PH1–PH4. The same direction holds on SZ-Taxi, where Mix reduces RMSE by 24.4%–25.2% relative to Physical.

The GCN family (Table 3) shows the same pattern (Los-loop PH1: GCN 8.14 ± 0.11 vs. graph-free baseline 4.88 ± 0.31). Figures 3 and 4 summarize five-seed mean RMSE for the T-GCN and GCN families separately. Figure 5 contrasts the dense physical adjacency with the much sparser multi-lag union. Throughout the tables, the italic *graph-free baseline* (identity adjacency) is a reference control, not a proposed method: it shows how much error remains when no spatial graph is used, and is not the primary comparison for the learned variants.

Table 3: GCN family: test RMSE (mean, sample standard deviation) over five seeds. GCN-MultiGSL consumes the union of the three multi-lag graphs as one static adjacency.

Method	PH1	PH2	PH3	PH4
<i>Los-loop (5 min)</i>				
<i>Graph-free baseline (NoSpatial)</i>	<i>4.88 (0.31)</i>	<i>5.60 (0.39)</i>	<i>6.02 (0.28)</i>	<i>6.26 (0.26)</i>
GCN (Physical)	8.14 (0.11)	8.54 (0.28)	8.76 (0.26)	8.76 (0.29)
GCN-GSL	7.83 (0.11)	8.48 (0.21)	8.45 (0.19)	8.93 (0.12)
GCN-cGSL	5.76 (0.20)	6.33 (0.32)	6.68 (0.25)	6.88 (0.21)
GCN-MultiGSL	9.78 (0.14)	10.05 (0.16)	10.24 (0.13)	10.27 (0.11)
<i>SZ-Taxi (15 min)</i>				
<i>Graph-free baseline (NoSpatial)</i>	<i>4.11 (0.00)</i>	<i>4.15 (0.00)</i>	<i>4.19 (0.00)</i>	<i>4.22 (0.00)</i>
GCN (Physical)	5.96 (0.01)	5.98 (0.00)	5.99 (0.00)	6.00 (0.00)
GCN-GSL	4.88 (0.01)	4.91 (0.00)	4.93 (0.01)	4.96 (0.00)
GCN-cGSL	4.64 (0.00)	4.67 (0.00)	4.70 (0.00)	4.73 (0.00)
GCN-MultiGSL	4.82 (0.01)	4.85 (0.00)	4.88 (0.01)	4.90 (0.00)

5.2 Contemporaneous Learned Graphs (GSL and cGSL)

Replacing the physical adjacency with a single contemporaneous DAGMA graph consistently reduces error relative to Physical. On Los-loop PH1, T-GCN-GSL and T-GCN-cGSL obtain 5.86 ± 0.21 and 5.82 ± 0.23 — about 26% lower RMSE than T-GCN (Physical, 7.88 ± 0.28). On SZ-Taxi the same pattern holds (about 21%–22% relative to Physical at PH1). For GCN, cGSL is clearly better than directed GSL on Los-loop (5.76 vs. 7.83) and both improve on the physical GCN (8.14). Among the two contemporaneous variants, T-GCN-GSL and T-GCN-cGSL are nearly indistinguishable; on GCN, symmetrization (cGSL) is the stronger of the two. Neither contemporaneous graph matches the multi-lag results discussed next.

5.3 Is the Gain Only Sparsity?

The physical graph is dense (order 10^3 off-diagonal edges on Los-loop), while single contemporaneous GSL uses 28 edges and the multi-lag construction uses 30 lag slots (union 28). To test whether any gain from a learned sparse graph is only an edge-count effect, we compare graphs at a matched 30-edge budget on Los-loop PH1 (Table 4; Figure 7).

At this matched budget, a random 30-edge graph is worse than using no graph (RandTop30 6.10 ± 0.12 vs. graph-free baseline 5.25 ± 0.19), and a top-correlation 30-edge graph is only slightly better than random but still worse than the graph-free baseline (CorrTop30 5.39 ± 0.10). The DAGMA multi-lag graphs with the same 30-edge budget, when used with lag-aligned T-GCN (MultiGSL) or gating (Mix), beat NoSpatial. Sparsity alone therefore does not explain the multi-lag result on this cell;

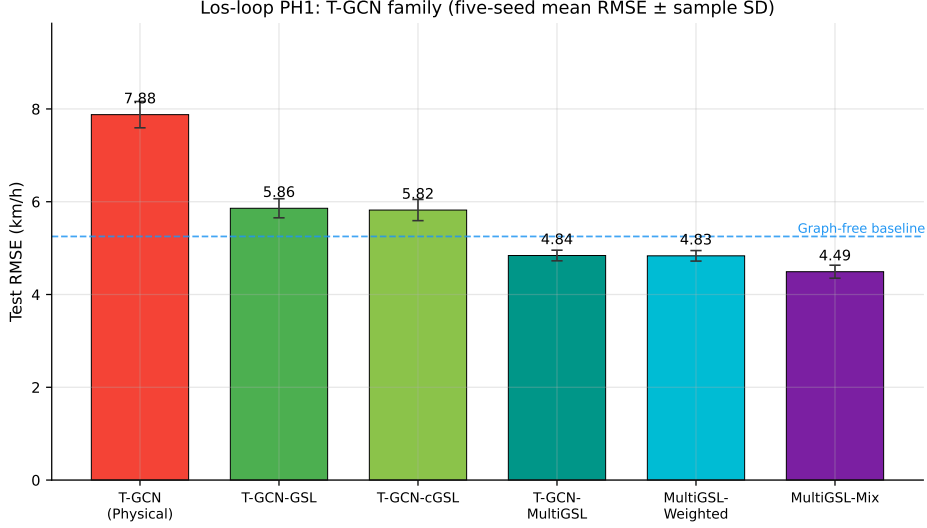


Fig. 3: Los-loop PH1: T-GCN family, five-seed mean RMSE (whiskers: sample SD). Dashed line: graph-free baseline.

the placement of the learned edges (and how they are used) matters. A capacity-matched graph-free model (hidden 74; single seed) stays near 5.14 RMSE, so the gate’s extra parameters do not explain Mix’s gain.

Extending the control across budgets (Table 5; Figure 6) strengthens this conclusion: for every $K \in \{10, 20, 30, 50, 80\}$, both RandTopK and CorrTopK remain at or above the graph-free baseline. Random graphs worsen monotonically with K ($5.37 \rightarrow 7.03$); correlation graphs are closest to the baseline at small K and also worsen as more edges are added ($5.27 \rightarrow 5.64$). In contrast, at the same 30-edge budget the DAGMA multi-lag graphs with lag-aligned use reach 4.84 (MultiGSL) and 4.49 (Mix). Increasing edge count alone does not recover the multi-lag gain; learned placement and lag-aligned use do.

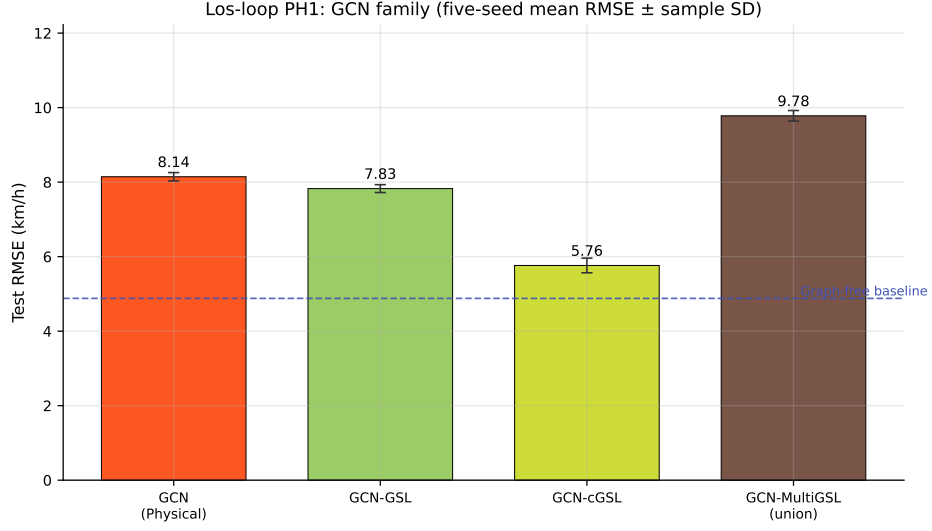


Fig. 4: Los-loop PH1: GCN family, five-seed mean RMSE (whiskers: sample SD). Dashed line: graph-free baseline (NoSpatial is not drawn as a bar).

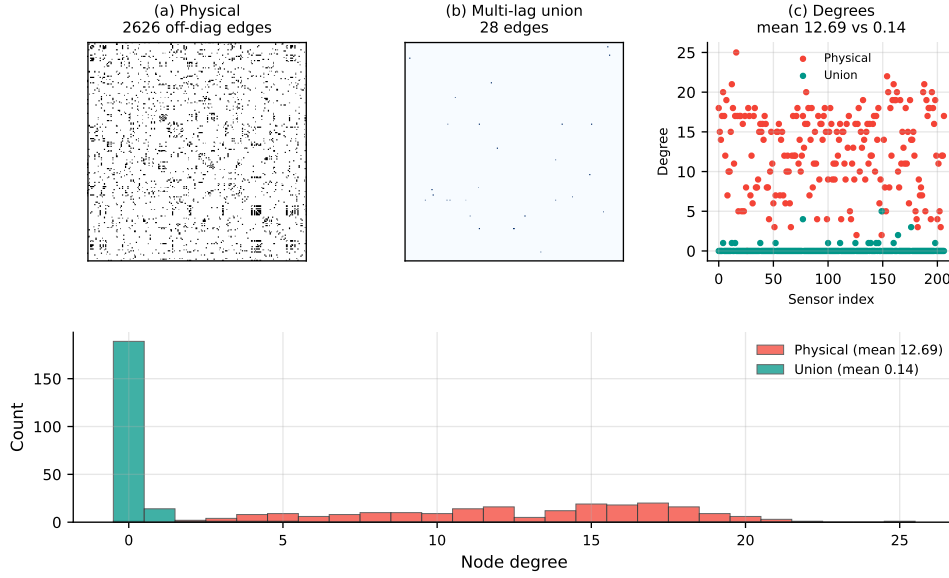


Fig. 5: Los-loop graph structure: (a) physical adjacency (2626 off-diagonal edges; mean degree 12.69); (b) multi-lag union after consumer thresholding (28 edges; mean degree 0.14); (c) node degrees. Learned graphs are statistical dependency estimates, not causal maps.

Table 4: Matched-sparsity and capacity controls on Los-loop at PH1. Upper block: graphs at a common 30-edge budget (or identity for the graph-free control), five seeds; lower block: capacity check on the graph-free model vs Mix, single seed (seed 42). Lower RMSE is better; bold marks the lowest mean among the 30-edge DAGMA variants in the upper block (Mix). Positive percentages in the last column mean worse than the graph-free baseline; negative mean better.

Configuration	RMSE	Budget / params	vs graph-free baseline
RandTop30	6.10 (0.12)	30 random edges	+16.1%
CorrTop30	5.39 (0.10)	30 top- Pearson	+2.6%
Graph-free baseline (NoSpatial)	5.25 (0.19)	identity	—
T-GCN-MultiGSL	4.84 (0.11)	30 DAGMA lag slots	−7.8%
T-GCN-MultiGSL-Mix	4.49 (0.14)	same graphs, gated	−14.5%
Graph-free $h=64$ (seed 42)	5.14	12,672 params	—
Graph-free $h=74$ (seed 42)	5.14	16,872 params	$\approx 0\%$
Mix (seed 42)	4.46	17,091 params	−13.3%

Table 5: Edge-budget sweep on Los-loop PH1 (five-seed mean $\pm$ sample SD; *lower RMSE is better*). RandTopK and CorrTopK are T-GCN with K random or top-correlation directed edges. The italic first row is the graph-free control. The last two rows are the main-experiment MultiGSL/Mix at the canonical $K = 30$ budget. Δ_{rel} is the relative RMSE difference vs the graph-free baseline (positive = higher error than the baseline; negative = lower error). Bold marks Mix, the lowest mean in the table.

Configuration	RMSE	Δ_{rel} vs graph-free
<i>Graph-free baseline</i>	5.23 ± 0.10	—
CorrTop10	5.27 ± 0.10	+0.3%
CorrTop20	5.29 ± 0.10	+1.1%
CorrTop30	5.39 ± 0.10	+3.0%
CorrTop50	5.52 ± 0.10	+5.4%
CorrTop80	5.64 ± 0.09	+7.7%
RandTop10	5.37 ± 0.17	+2.6%
RandTop20	5.80 ± 0.13	+10.8%
RandTop30	6.10 ± 0.12	+16.5%
RandTop50	6.50 ± 0.10	+24.3%
RandTop80	7.03 ± 0.15	+34.4%
T-GCN-MultiGSL ($K=30$)	4.84 ± 0.11	−7.5%
T-GCN-MultiGSL-Mix ($K=30$)	4.49 ± 0.14	−14.2%

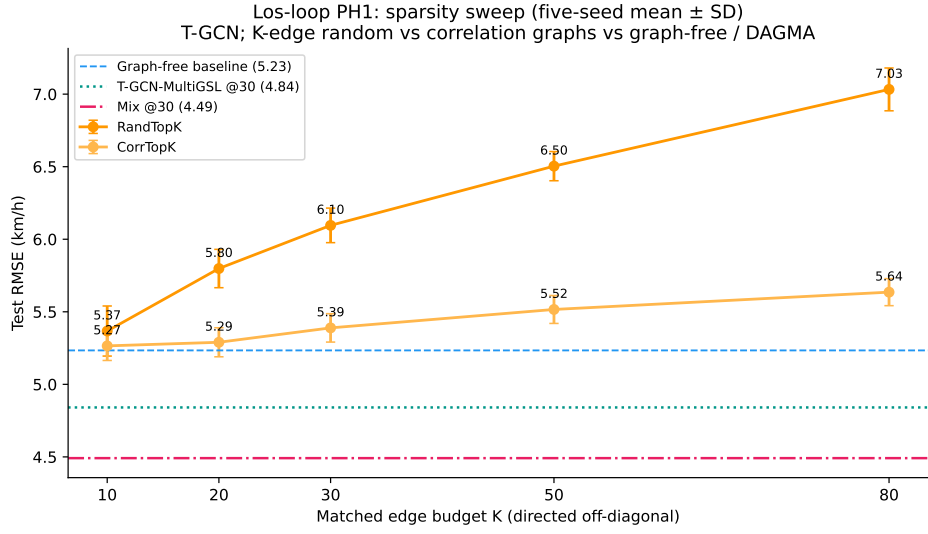


Fig. 6: Edge-budget sweep on Los-loop PH1 (five-seed mean $\pm$ sample SD). As K increases, random graphs degrade rapidly and correlation graphs degrade more slowly; neither family reaches the main-experiment MultiGSL/Mix reference lines at the same 30-edge budget.

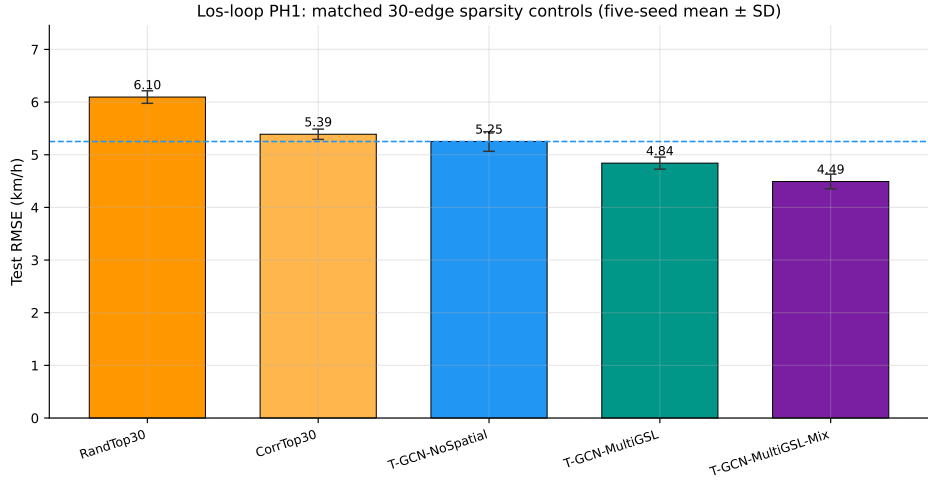


Fig. 7: Matched-sparsity controls on Los-loop at PH1 (five-seed mean $\pm$ sample SD). RandTop30 and CorrTop30 use a 30-edge budget; MultiGSL and Mix use the DAGMA multi-lag graphs. Dashed line: graph-free baseline.

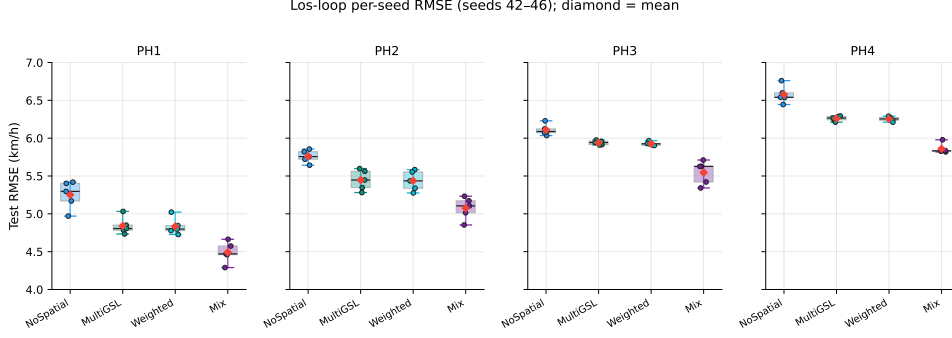


Fig. 8: Los-loop per-seed test RMSE for the multi-lag use ladder (seeds 42–46). Diamonds mark five-seed means matching Table 2.

5.4 Multi-Lag Learned Graphs

On Los-loop the multi-lag graphs are informative (12/3/15 edges by lag; union 28). T-GCN-MultiGSL yields 4.84 ± 0.11 at PH1; Mix improves to 4.49 ± 0.14 . Relative to the graph-free baseline, Mix reduces RMSE by 14.5%, 11.9%, 9.2%, and 10.9% at PH1–PH4 (Table 2; 5/5 paired seeds). Relative to Physical T-GCN: 43.0%, 37.6%, 33.7%, 32.3%. Paired t -tests for Mix vs graph-free baseline give $p \approx 0.0015$, 0.0015, 0.0008, $< 10^{-4}$ (reference only; Section 4.7).

Figure 8 shows the per-seed distributions: Mix has the lowest mean RMSE among the evaluated configurations on Los-loop at all four horizons. Weighted is essentially tied with the fixed MultiGSL assignment (differences ≤ 0.01 RMSE).

When the same multi-lag artifacts are merged into one static union for GCN-MultiGSL, error is 9.78 ± 0.14 on Los-loop PH1 — worse than the physical GCN and more than twice T-GCN-MultiGSL on the same graphs (Table 3). The union retains 28 of 30 lag slots; the comparison does not isolate consumption from backbone differences, but shows that multi-lag graphs are not automatically beneficial once estimated.

5.5 Dataset Dependence and Temporal Resolution

Figure 10 shows an analogous predicted-versus-actual overlay on SZ-Taxi: Physical and Mix, matching the aggregate results.

On SZ-Taxi the multi-lag fit retains only two lagged edges. Mix vs NoSpatial differs by at most 0.015 RMSE ($\leq 0.34\%$). The Los-loop result does not transfer under this construction.

Los-loop at 15-minute sampling (resolution control).

To test whether SZ-Taxi’s weak separation could be an artifact of coarser sampling, Los-loop was resampled to 15-minute steps. PH1–4 then correspond to 15, 30, 45, and 60 minutes ahead (wall-clock equivalent to PH3/6/9/12 on the 5-minute grid, but not the same as direct PH3/6/9/12 evaluation on 5-minute data). Mix vs graph-free baseline: 27.4%, 21.4%, 19.8%, 16.1% relative reductions with 5/5 seed wins (Table 6).

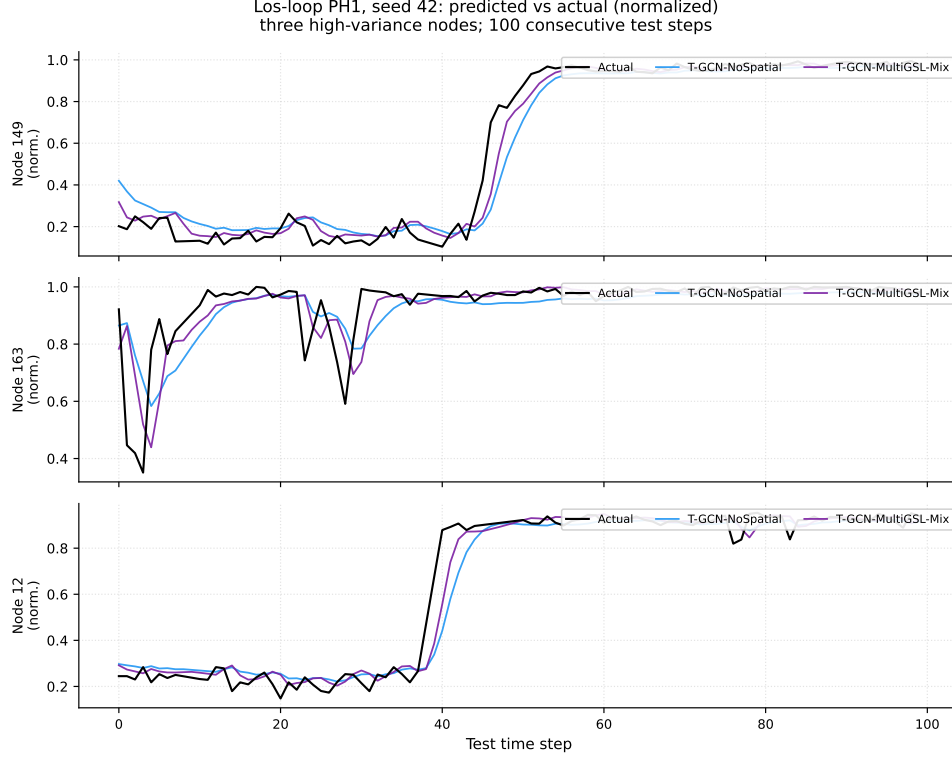


Fig. 9: Predicted versus actual speed on Los-loop (PH1, seed 42) for three high-variance nodes (100 test steps). Each row overlays ground truth with T-GCN (Physical) and T-GCN-MultiGSL-Mix (normalized scale).

Temporal resolution alone does not explain SZ-Taxi; lag-structure recovery differs sharply (30 vs. 2 slots).

5.6 Summary of Findings

1. Learning $\mathbf{A}$ from data can improve on the physical graph, but a graph-free control is necessary to judge the contribution of spatial structure.
2. A single contemporaneous learned graph does not beat the graph-free control on these benchmarks.
3. Multi-lag graphs help on Los-loop when used in a lag-aligned way in T-GCN; Mix has the lowest mean RMSE among evaluated multi-lag variants. Sparsity alone does not explain this gain on the matched-budget cell.
4. The benefit is dataset-dependent: SZ-Taxi shows little separation from the graph-free baseline.

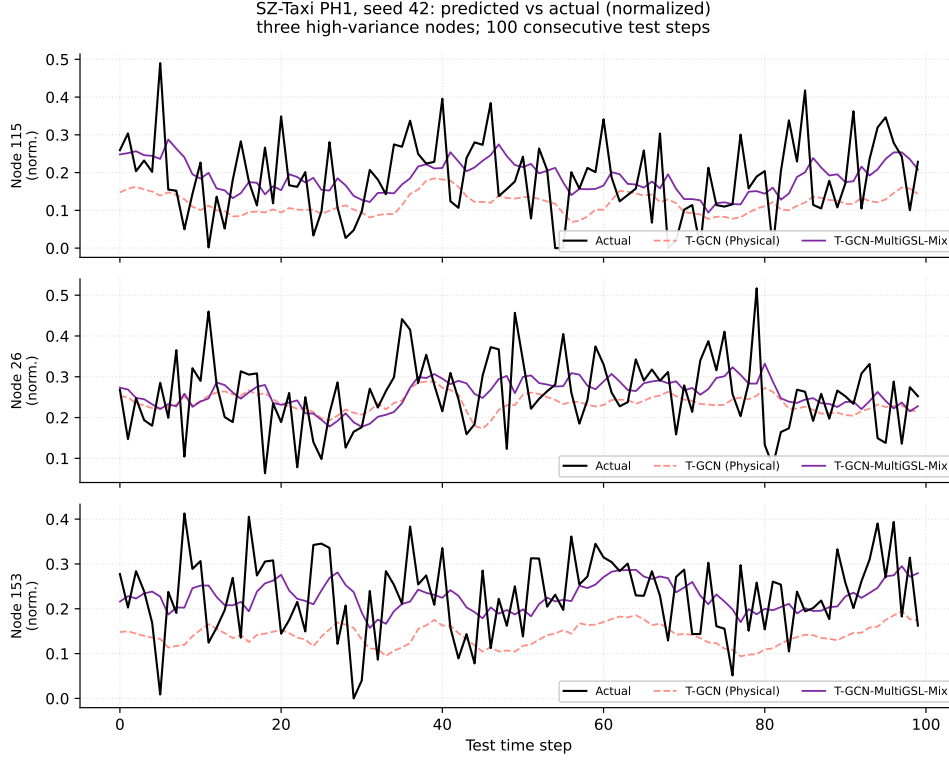


Fig. 10: Predicted versus actual speed on SZ-Taxi (PH1, seed 42) for three high-variance nodes (100 test steps). Layout as in Figure 9. Predictions of the graph-free baseline and Mix nearly overlap, consistent with the aggregate RMSE separation on this dataset.

6 Discussion

6.1 Learned Structure versus the Physical Graph

The primary comparison in this study is whether a learned adjacency improves on the conventional physical road-network graph. On both benchmarks it does, for most learned configurations. On Los-loop PH1, contemporaneous GSL/cGSL already reduce RMSE by roughly a quarter relative to T-GCN (Physical), and multi-lag use reduces error by about 43% relative to Physical at PH1 (and 32–38% at PH2–PH4). On SZ-Taxi the same direction holds, with Mix about 24–25% below Physical.

The physical graph is also dense (order 10^3 off-diagonal entries on Los-loop). Degradation relative to a model with no spatial graph is consistent with excessive neighborhood mixing or an unsuitable inductive bias; we do not measure oversmoothing directly. That observation belongs to how GCN and T-GCN use a dense proximity matrix and can be studied independently of GSL. We include a graph-free identity

Table 6: Los-loop temporal-resolution variant at 15-minute sampling. PH k denotes 15 k minutes ahead; not comparable to PH k of the 5-minute series. Five seeds; sample SD (ddof= 1).

Method	PH1 (15 min)	PH2 (30 min)	PH3 (45 min)	PH4 (60 min)
<i>Graph-free baseline</i>	<i>8.600 (0.279)</i>	<i>9.311 (0.220)</i>	<i>10.048 (0.083)</i>	<i>10.460 (0.220)</i>
T-GCN-MultiGSL	7.246 (0.333)	8.571 (0.365)	9.112 (0.087)	9.756 (0.234)
T-GCN-MultiGSL-Mix	6.240 (0.209)	7.322 (0.378)	8.063 (0.227)	8.777 (0.364)
Mix vs graph-free	−27.4%	−21.4%	−19.8%	−16.1%
Mix wins (of 5)	5	5	5	5

control only as a reference: it shows the error level when no spatial graph is used, and is not the primary target of the proposed methods.

Learned contemporaneous graphs sit between Physical and the multi-lag variants: they improve on the physical graph but are weaker than the multi-lag results on Los-loop. What they provide is a less harmful, much sparser neighborhood than the dense proximity graph.

6.2 When Learned Structure Helps

The strongest Los-loop result is multi-lag DAGMA graphs used in a lag-aligned way inside T-GCN. Three observations support a structure-learning interpretation rather than a pure sparsity interpretation. First, at a matched 30-edge budget, random and correlation-placed graphs are worse than the graph-free reference, while DAGMA-placed lag graphs with lag-aligned use are better (Los-loop PH1 only). Second, the multi-lag construction recovers substantially more lagged structure on Los-loop than on SZ-Taxi (30 vs. 2 slots). Third, the same multi-lag artifacts fail when merged into a static union for GCN, so estimation alone is not enough; the backbone must use the structure in a way that matches how it was organized (lag blocks).

The GCN/T-GCN contrast for contemporaneous graphs (cGSL helps GCN more than T-GCN) is an aggregation-compatibility observation of this study. We do not interpret the contemporaneous DAG as a temporal graph (Section 3.4); temporal organization of learned structure is carried by the multi-lag construction only.

6.3 What the Learned Graph Represents

We treat the learned adjacencies as *statistical dependency* estimates under a linear DAGMA fit. The acyclicity constraint is used as an optimization device for the estimator; we do not interpret the fitted matrices as causal traffic mechanisms. The lag-structured reading of the multi-lag blocks is consistent with the input construction $Z = [x(t-3), \dots, x(t)]$ and with the consumption experiments, but it is not independently validated against ground-truth propagation. The gated variant changes how

static lag graphs are used per node and timestep; it does not produce a time-varying edge set.

6.4 Dataset Dependence

SZ-Taxi and Los-loop differ in sampling interval, sensor count, and road environment. The 15-minute Los-loop control (Section 5.5) shows that coarser sampling alone does not suppress the multi-lag gain. The more concrete difference in these experiments is how much structure the estimator recovers: a nearly empty multi-lag fit on SZ-Taxi is associated with results close to the graph-free baseline. This is a scope condition on GSL for traffic, not a failure of the general idea that adjacency can be estimated from data.

7 Limitations

- Statistical power. Main results use five training seeds. Sample standard deviations and paired win counts are reported; with $n = 5$ the exact Wilcoxon test cannot fall below $p = 0.0625$, so we avoid definitive significance claims.
- Control scope. Matched-sparsity and capacity controls exist only for Los-loop at PH1. The edge-budget sweep covers $K \in \{10, 20, 30, 50, 80\}$ for random and correlation graphs only; the DAGMA multi-lag graphs are evaluated at the canonical 30-edge budget (and in the main results matrix). There is no sparsified physical graph, and no systematic sensitivity sweep over the DAGMA sparsity coefficient λ_1 or over support thresholds was performed. In the contemporaneous construction λ_1 is 0.02 (Los-loop) and 0.01 (SZ-Taxi); in the multi-lag construction it is 0.01 on both datasets. These values follow the original GSL protocol and are distinct from the internal and consumer thresholds and from the forecasting-loss regularization. A systematic study of λ_1 remains future work.
- Graph-estimation cost. The main computational consideration is the offline DAGMA graph-estimation stage rather than repeated forecasting inference. In our runs, a contemporaneous fit on the order of 10^2 sensors is on the order of tens of minutes per horizon, while the 828-variable multi-lag fit is on the order of hours on CPU. The learned graph is estimated once per dataset/construction and reused for all forecasting seeds, epochs, and test evaluation, so this cost is not repeated inside the forecasting loop. Scalability to substantially larger road networks remains an open question.
- Estimator assumptions. Linear DAGMA with ℓ_2 score and acyclicity regularizer; nonlinear dependency and misspecified lag depth $L = 3$ are not examined.
- Static graphs. Learned adjacencies are fitted once. Mix varies *use* of those graphs, not the edge set; time-varying structure learning remains future work.
- Backbones. Evaluation uses GCN and T-GCN as relatively established, representative spatial and spatiotemporal backbones. This choice provides a controlled setting for comparing graph construction and graph use; transfer to newer attention-based, adaptive-graph, or transformer-style architectures is an open empirical question. The comparison between static-union GCN and lag-specific T-GCN does not isolate incorporation strategy independently of the backbone.

8 Conclusion and Future Works

This paper revisits graph structure learning for traffic forecasting under a protocol that includes physical, graph-free, contemporaneous learned, and multi-lag learned adjacencies on two public benchmarks. The main findings are:

1. Learning the adjacency from traffic data can reduce forecasting error relative to the conventional physical road-network graph. On Los-loop, multi-lag DAGMA graphs with lag-aligned use in T-GCN reduce RMSE by up to 43.0% relative to Physical at PH1–PH4; on SZ-Taxi the reduction versus Physical is about 24–25%, with smaller absolute gains.
2. A single contemporaneous DAGMA graph improves on Physical in many settings but is weaker than the multi-lag results on Los-loop.
3. Multi-lag learned graphs are most useful on Los-loop when used in a lag-aligned way inside T-GCN; per-node gating (T-GCN-MultiGSL-Mix) has the lowest mean RMSE on Los-loop among the evaluated configurations (14.5% relative RMSE reduction vs. the graph-free baseline at PH1). Matched-budget controls show that sparsity alone does not explain this gain on the studied cell.
4. The benefit is dataset-dependent. On SZ-Taxi, where the multi-lag fit is nearly empty, learned variants remain close to the graph-free baseline. A 15-minute resampling of Los-loop retains a large relative gain, so coarser sampling alone does not explain the SZ-Taxi result.
5. Learned graphs are interpreted as statistical dependency structure, not causal maps of traffic influence.

Future work includes longer horizons at 5-minute resolution, additional networks, sparsified-physical controls and λ /threshold sensitivity, other backbones, hybrid combinations with attention [8], and genuinely time-varying graph estimation (e.g. incremental or sliding-window structure learning) rather than gated use of a fixed graph. Overall, learning the adjacency from traffic data can improve short-horizon forecasting when the recovered structure is informative and is used in a manner consistent with how it was estimated.

Appendix A Additional Experimental Results

A.1 Lag ablation (supplementary, single seed)

Figure A1 summarizes which lag-specific multi-lag graphs are used when only a subset of lags is retained (Los-loop PH1, seed 42, same training protocol as the main experiments but a single seed and outside the five-seed main matrix). Each individual lag improves on T-GCN-NoSpatial by about 10% relative RMSE; the full three-lag set is best in this ablation (about 13.3% relative). These numbers are supplementary diagnostics for lag complementarity and are not a substitute for the five-seed results in Tables 2 and 3.

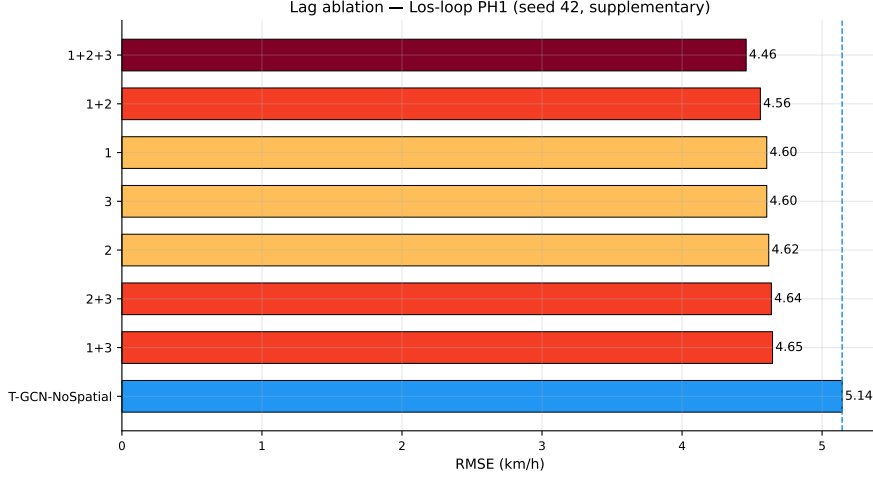


Fig. A1: Supplementary lag ablation (Los-loop PH1, seed 42). Bars are test RMSE for subsets of the three multi-lag graphs under the gated mechanism (T-GCN-MultiGSL-Mix family), with edge counts in parentheses. The dashed line is T-GCN-NoSpatial. Single seed; see Section A.1.

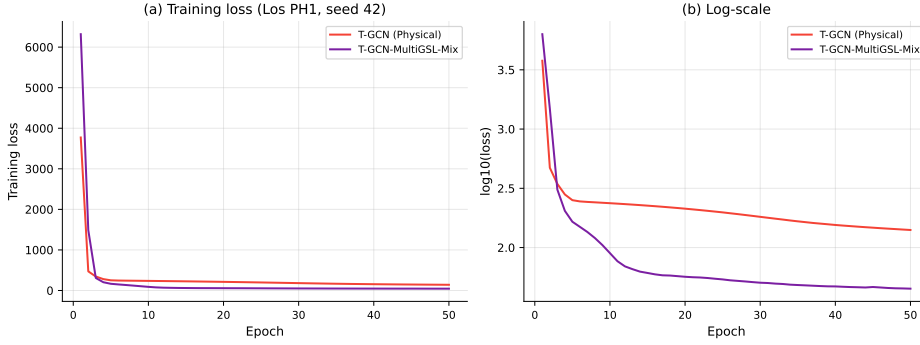


Fig. A2: Compact training-loss histories for T-GCN (Physical) and T-GCN-MultiGSL-Mix (Los-loop PH1, seed 42). Left: raw loss; right: log-scale. Replaces dense multi-panel convergence grids; not primary evidence for method ranking.

A.2 Training curves

Figure A2 shows compact training-loss histories from checkpoint runs (Los-loop PH1, seed 42), comparing the physical road-network model (T-GCN) with the proposed gated multi-lag model (T-GCN-MultiGSL-Mix). They are included for reproducibility context only; main comparisons use final test RMSE with five-seed statistics (Section 4.7), not per-epoch training curves.

Appendix B Bibliometric Analysis Methodology

To support the motivating observation in the Introduction (the rise of graph-based traffic forecasting alongside continued reliance on heuristic graph construction), we conducted a systematic bibliometric analysis. This appendix is independent of the forecasting experiments.

A broad Scopus search with the keyword “traffic forecast*” retrieved on the order of 60,000 documents. Restricting to peer-reviewed English articles in Engineering or Computer Science yielded a core collection of 784 articles published between 2000 and 2025.

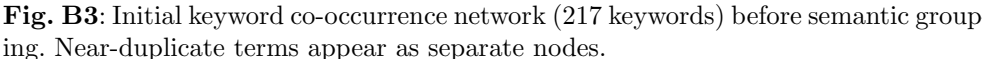
The raw export contained 6,343 unique author and index keywords. After removing terms that appear fewer than 10 times, 217 keywords remained. An initial VOSviewer co-occurrence network (Figure B3) showed near-duplicate labels (e.g. “graph neural network”, “GNN”, “graph convolution network”) as separate nodes, which obscured broader patterns.

To reduce this fragmentation we applied a cleaning pipeline implemented in the project repository: (i) preprocessing (case folding, removal of special characters and stop words); (ii) expansion of 30 common abbreviations to their full forms; and (iii) semantic grouping with Sentence-BERT embeddings. The process condensed the 217 terms into 66 concept groups, each labeled by a representative term. Figure B4 shows word clouds for three illustrative clusters.

The refined co-occurrence network (Figure B5) shows that graph-based methods are among the most recent methodological foci in this literature (nodes colored by average publication year), while many such studies still construct the adjacency from road topology or other heuristics. This pattern motivates studying graph structure learning for traffic forecasting; it does not by itself evaluate any forecasting method.

References

- [1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. *Journal of Advanced Transportation* **2021** (2021)
- [2] Ye, J., Zhao, J., Ye, K., Xu, C.: How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Transactions on Intelligent Transportation Systems* (2020)
- [3] Xie, P., Li, T., Liu, J., Du, S., Yang, X., Zhang, J.: Urban flow prediction from spatiotemporal data using machine learning: A survey. *Information Fusion* **59**, 1–12 (2020)
- [4] Chen, F., Zhang, Y., Chen, L., Meng, X., Qi, Y., Wang, J.: Dynamic traveling time forecasting based on spatial-temporal graph convolutional networks. *Front. Comput. Sci.* **17**(6) (2023) <https://doi.org/10.1007/s11704-023-2704-x>
- [5] Zhou, Z., Basker, R., Yeung, D.-Y.: Graph neural networks for multivariate time-series forecasting via learning hierarchical spatiotemporal dependencies.



- [6] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Trans. on Intell. Transportation Systems* **21**(9), 3848–3858 (2020)
- [7] Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H.: A3t-gcn: Attention temporal graph convolutional network for traffic forecasting. *ISPRS International Journal of Geo-Information* **10**(7) (2021) <https://doi.org/10.3390/ijgi10070485>
- [8] Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based graph neural networks: a survey. *Artificial Intelligence Review* **56**(2), 2263–2310 (2023) <https://doi.org/10.1007/s10462-023-10577-2>
- [9] Chen, G., Chen, L., Li, Z., Wei, Y., He, Z., Yang, F.: Ta-stgcn: trend-aware attention spatio-temporal graph convolutional network for traffic flow prediction.

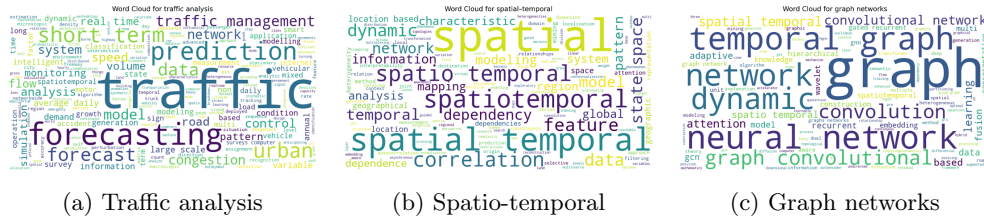


Fig. B4: Word clouds for three representative bibliometric clusters. Node size reflects frequency within the cluster.

International Journal of Data Science and Analytics **22**(1), 56 (2026) <https://doi.org/10.1007/s41060-026-01040-w>

- [10] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: NeurIPS (2018)
- [11] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22. Curran Associates Inc., Red Hook, NY, USA (2024)
- [12] Hamed, M.M., Al-Masaeid, H.R., Said, Z.M.B.: Short-term prediction of traffic volume in urban arterials. *Journal of Transportation Engineering* **121**(3), 249–254 (1995)
- [13] Fabian, X., Ban, G., Boussaïd, R., *et al.*: Modeling and forecasting vehicular traffic flow as a seasonal arima process. *Journal of Transportation Engineering* **129**(6), 664–672 (2003)
- [14] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. *Statistics and Computing* **14**(3), 199–222 (2004)
- [15] Lippi, M., Bertini, M., Frasconi, P.: Short-term traffic flow forecasting. *IEEE Transactions on Intelligent Transportation Systems* **14**(2), 871–882 (2013)
- [16] Ojeda, L.L., Kibangou, A.Y., Wit, C.C.: Adaptive kalman filtering for multi-step ahead traffic flow prediction. In: American Control Conference (2013)
- [17] Park, D., Rilett, L.R.: Forecasting freeway link travel times with a multilayer feed-forward neural network. *Computer-Aided Civil and Infrastructure Engineering* **14**(5), 357–367 (2010)
- [18] Zhang, J., Zheng, Y., Qi, D.: Deep spatio-temporal residual networks for citywide crowd flows prediction, 1655–1661 (2017)
- [19] Cao, X., Zhong, Y., Zhou, Y., Wang, J., Zhu, C., Zhang, W.: Interactive temporal

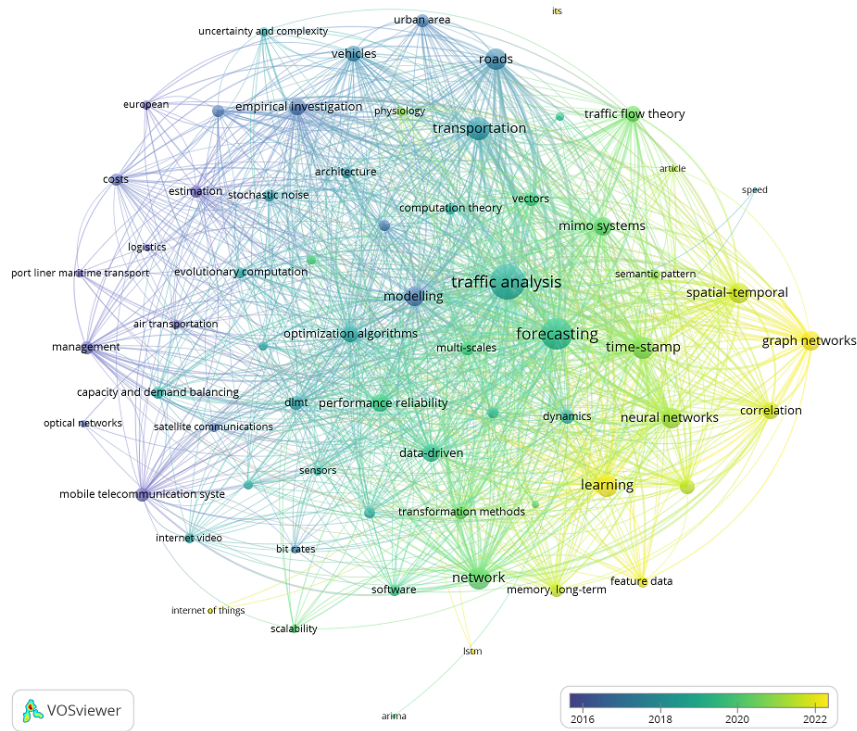


Fig. B5: Clustered keyword co-occurrence network from 784 traffic forecasting articles (2000–2025), generated with semantic similarity grouping. Nodes represent keyword clusters (minimum 10 occurrences), colored by average publication year.

recurrent convolution network for traffic prediction. *IEEE Access* **5**, 9893–9901 (2017)

- [20] Fu, R., Zhang, Z., Li, L.: Using lstm and gru neural network methods for traffic flow prediction. In: Youth Academic Conference of Chinese Association of Automation, pp. 324–328 (2017)
- [21] Khan, H.M., Khan, A., Villar, S.G., Lopez, L.A.D., Almaleh, A., Al-Qahtani, A.M.: A comparative study of optimized-lstm models using tree-structured parzen estimator for traffic flow forecasting in intelligent transportation. *Computers, Materials and Continua* **83**(2), 3369–3388 (2025) <https://doi.org/10.32604/cmc.2025.060474>
- [22] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: ICLR (2017)

- [23] Zuo, J., Zeitouni, K., Taher, Y., Garcia-Rodriguez, S.: Graph convolutional networks for traffic forecasting with missing values. *Data Mining and Knowledge Discovery* **37**(2), 913–947 (2023) <https://doi.org/10.1007/s10618-022-00903-7>
- [24] Wang, B., Long, Z., Sheng, J., Zhong, Q.: Spatial-temporal similarity fusion graph adversarial convolutional networks for traffic flow forecasting. *Journal of the Franklin Institute* **361**(17), 107299 (2024) <https://doi.org/10.1016/j.jfranklin.2024.107299>
- [25] Xia, Z., Zhang, Y., Yang, J., Xie, L.: Dynamic spatial-temporal graph convolutional recurrent networks for traffic flow forecasting. *Expert Systems with Applications* **240**, 122381 (2024) <https://doi.org/10.1016/j.eswa.2023.122381>
- [26] Remmouche, B., Boukraa, D., Zakharova, A., Bouwmans, T., Taffar, M.: Long-term spatio-temporal graph attention network for traffic forecasting. *Expert Systems with Applications* **288**, 128244 (2025) <https://doi.org/10.1016/j.eswa.2025.128244>
- [27] Zeb, A., Zheng, J., Ye, Y., Chen, J., Zhang, S., Wei, X., Jianqiao Yu, J.: Traffic forecasting with meta attentive graph convolutional recurrent network. *Expert Systems with Applications* **287**, 128073 (2025) <https://doi.org/10.1016/j.eswa.2025.128073>
- [28] Jia, C., Jiang, F., Chen, Z., Wang, R., Xiao, Y.: A dynamic traffic flow prediction method based on spatio-temporal graph transformer. *Applied Intelligence* **55**(16), 1048 (2025) <https://doi.org/10.1007/s10489-025-06895-3>
- [29] Liu, R., Shin, S.-Y.: A review of traffic flow prediction methods in intelligent transportation system construction. *Applied Sciences* **15**(7) (2025) <https://doi.org/10.3390/app15073866>
- [30] Peng, L., Liao, X., Li, T., Guo, X., Wang, X.: An overview based on the overall architecture of traffic forecasting. *Data Science and Engineering* **9**(3), 341–359 (2024) <https://doi.org/10.1007/s41019-024-00246-x>
- [31] Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., et al.: Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018)
- [32] Liu, H., Wei, J., Xu, T.: Community detection based on community perspective and graph convolutional network. *Expert Systems with Applications* **231** (2023) <https://doi.org/10.1016/j.eswa.2023.120748>
- [33] Liping Deng, B.G., Zheng, W.: GCN-based weakly-supervised community detection with updated structure centres selection. *Connection Science* **36**(1), 2291995 (2024) <https://doi.org/10.1080/09540091.2023.2291995>

- [34] Amintoosi, M.: Overlapping clusters in cluster graph convolutional networks. *Journal of Algorithms and Computation* **53**(2), 33–45 (2021)
- [35] Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., Leskovec, J.: Graph convolutional neural networks for web-scale recommender systems. In: *KDD* (2018)
- [36] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph Neural Networks: Graph Structure Learning*, pp. 297–321. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6054-2_14 . https://doi.org/10.1007/978-981-16-6054-2_14
- [37] Amintoosi, M.: Urban traffic prediction with learning based graph convolutional networks. In: *Proceedings of the 55th Annual Iranian Mathematics Conference*, Ferdowsi University of Mashhad, pp. 145–148 (2024). (In Persian). <https://github.com/mamintoosi/TGCN-GSL-TF>
- [38] Yu, B., Lee, Y., Sohn, K.: Forecasting road traffic speeds by considering area-wide spatio-temporal dependencies based on a graph convolutional neural network (gcn). *Transportation Research Part C: Emerging Technologies* **114**, 189–204 (2020) <https://doi.org/10.1016/j.trc.2020.02.013>